

Python Programming

Aj. info

Page No. _____

Date _____

Uses:

Web Development (Django, Flask)

Data Analysis (Pandas, Numpy)

Machine Learning and AI (Tensor Flow, PyTorch)

Automation and Scripting

Scientific Computing

Game Development

For vs Code
name.py.

Ex: Factorial of a number

n=5

fact=1

for i in range(1, n+1):

fact *= i

print(fact)

Ex: Print Table of two no.s

n=5

for i in range(1, 11):

print(n, "x", i, "=", n*i)

Ex: Swap two numbers

a=10

b=20

a, b = b, a

print(a, b)

Ex: Simple function

```
def add(a, b):  
    return a + b  
print(add(5, 3))
```

Ex: Even or odd

```
a = 8  
if (a % 2 == 0):  
    print("Even")  
else:  
    print("odd")
```

Ajanta

Page No.
Date

Ex: Input two no.s and find sum

```
a = int(input("Enter first number:"))  
b = int(input("Enter second number:"))  
print("Sum = ", a + b)
```

Ex: Output using f-string

```
name = "Ajay"  
age = 20  
print(f"Name : {name} ; Age : {age}")
```

Ex: Avg. marks and grade

```
p = int(input("Enter marks in physics?"))  
c = int(input("Enter marks in chemistry?"))  
m = int(input("Enter marks in maths?"))  
avg = (p + c + m) / 3
```

```
if (avg >= 90):  
    print('Result first')  
elif (avg >= 80 and avg < 90):  
    print('Result second')  
elif (avg >= 70 and avg < 80):  
    print('Result third')  
else:  
    print('fail')
```

Data types in Python

```
age = 20 # integer  
name = "Ajay" # string  
cgpa = 9.82 # float  
a = True, b = False (Boolean)
```

Rule for defining variable in Python.

- * Variables names must start with letter (a-z, A-Z) or an underscore (_).
- * They can contain letters, numbers and underscores.
- * Variables names are case sensitive (age & Age are diff).
- * Avoid using Python keywords (e.g. if, for, while) as variable name.

```
# 34age = 4 (Invalid)  
age = 4 (valid)  
a $$ age = 4 (Invalid)
```

```
age = 3  
print(age)  
print(type(age))
```

To find data type

Built in data types

Lists = Ordered, mutable collections eg:- [1, 2, 3]

Tuples = Ordered, immutable collections eg:- (1, 2, 3)

Sets = unordered collection of unique elements eg:- {1, 2}

Dictionaries = key-value pairs eg:- {"Name": "Ajay", "age": 20}

Type Casting in Python → Process of converting one data type to another.

* a = 24

b = "24"

print(a)

print(type(a)) → int

print(b)

print(type(b)) → string.

c = int(b) # Convert b to an integer

print(c)

print(type(c)) → int

d = 22.3

e = str(d)

print(e)

print(type(e)).

* # Convert string to integer.

num_str = "10"

num_int = int(num_str)

print(num_int) → Output: 10

* # Convert integer to string

num = 25

num_str = str(num)

print(num_str) → Output: "25"

* # Convert float to integer.

pi = 3.14

pi_int = int(pi)

print(pi_int).

Comments in Python

→ used for single line comments.

""" """ → Multi line comments.

Ctrl + / → used to convert into comments.

Escape Sequences → used to include special characters in strings.

Common escape sequences:-

\n: newline

\t: tab

\\: Backslash → used to print single backslash.

\": Double quote

\': single quote.

print("Hey how are you\n I am good \n newline")

print("Hello \" World")

print("This is a tab\t character").

sep = " " → used to separate multiple arguments in print statement

end = "\n" → use for new line in multiple arguments.

end = ".." → used to separate by .. for multiple argument.


```
print("Hello", "Ajay", "How are you")
```

```
print("Hello", "Ajay", "How are you", sep=" ")
```

```
print("Hello world", end="!\n")
```

```
print("Hello world", end="..").
```

REPL → Read evaluate Print Loop

Operators in Python

(i) Arithmetic operators

Addition (+), Subtraction (-), Multiplication (*), Division (/)

Modulus (%), Exponentiation (**), Floor Division (//)

a=34

b=2

```
print("a+b=", a+b) → 36
```

```
print("a-b=", a-b) → 32
```

```
print("a*b=", a*b) → 68
```

```
print("a/b=", a/b) → 17.0
```

```
print("a%b=", a%b) → 0
```

```
print("a//b=", a//b) → 17
```

```
print("a**b=", a**b) → 1156
```

(ii) Comparison operators

Equal (==), Not equal (!=), Greater than (>)

Less than (<), Greater than or equal (>=),

Less than or equal (<=)

```
print(10>5) # output: True
```

```
print(10==5) # output: False.
```

(iii) Logical operators and, or, not

```
print(True and False) # output: False
```

```
print(True or False) # output: True
```

```
print(not True) # output: False.
```

(iv) Assignment operators

=, +=, -=, *=, /=, %=, **=, //=

a x=10

```
x+=5 # Equivalent to x=x+5
```

```
print(x) # output: 15
```

(v) Membership operators:- in, not in.

```
fruits = ["apple", "banana", "cherry"]
```

```
print("banana" in fruits) # output: True.
```

(vi) Identity operators:- is, is not

x=10

y=10

```
print(x is y) # output: True.
```


* Print the poem Twinkle, twinkle little star,
How I wonder what you are!

→ print("Twinkle, twinkle little star, \n How I wonder what you are").

* name = "Ajayraj"

age = 21

height = 176.2

is_student = True

* Alt + Shift + Down Arrow

key

(To print same line again in code)

print("The name of student is " + name + ". He is" + str(age) + " years old." + " He is " + str(height) + " cms tall." + "\n Student: " + str(is_student)).

→ Used to concatenate strings. (not int / float).

* num = "45"

num_int = int(num)

print(num_int)

num_int += 10

print(num_int)

} Convert string into integers and add 10 and print result.

* Take input and print square and cube.

a = int(input("Enter your number \n"))

print("Square is", a**2)

print("Cube is", a**3)

if-else statements

* age = int(input("Enter your age?"))

if (age > 18):

print("You can drive")

else:

print("You cannot drive").

* age = int(input("Enter your age?"))

if (age > 18):

print("You can drive")

elif (age == 18):

print("Let's schedule an interview")

else:

print("You cannot drive").

Match case

a = int(input("Enter a number b/w 1 and 10:"))

match a:

case 1:

print("You won a charger")

case 2:

print("You won an ipad")

case 3:

print("You won a television")

case _:

print("Better luck next time"). } → for default case.

Loops in Python

(i) for loop: for i in range (1, 6):
print (i)

1 }
2 }
3 } output - # range function goes from
4 } 1 to 5 in this case.
5 }

for i in range (1, 11):
print (5*(i)) # print ("5", "x", i, "=", 5*(i))

(ii) while loop:

i = 1
while i < 6:
print (i)
i = i + 1 → # i + 1

* Ex. of infinite while loop:

while True:
print ("This will run
forever!")

i = 1
while True:
print (i)
i = i + 1

break statement → To cancel the execution of the loop.

for i in range (1, 21):
~~print (i)~~
if i == 11:
break
print (i)

output = 1
2
3
4
.
.
10
11

Continue statement

for i in range (1, 21):
if i == 10:

continue # continue the loop for the next
iteration here itself
print (i)

pass statement

for i in range (1, 11):

if i == 12:

pass # Do nothing
print (i)

(i) Write a program to check a no. is positive, negative or zero.

(ii) Even / odd program.

(iii) Ask the user to enter a day no. (1-7) and print the corresponding day of the week using match case.

(iv) Ask the user for two numbers and an operation (+, -, *, /)
Perform operation using match case.

num1 = int(input("Enter 1st num"))

num2 = int(input("Enter 2nd num"))

operation = input("Choose operation")

match operation:

case +:

print (num1 + num2)

case -:

case *:

case /:

(v) Calculate sum of num from 1 to 1000?

```
sum = 0
for i in range(1, 1001):
    print(i)
    sum += i # sum = sum + i
print(sum)
```



(vi) Print

```
*
**
***
****
*****
```

```
for i in range(1, 6):
    print("*" * i)
```

(vii) Print 1-100 using while and find sum.

```
sum = 0
i = 1
while i <= 100:
    print(i)
    sum = sum + i
    i = i + 1
print(sum)
```

(viii) Write a program that keeps asking the user to enter a password until they enter the correct one.

```
password = "y2k123"
entered_pass = input("Enter password:")

while (entered_pass != password):
    entered_pass = input("Wrong! Try again and enter password:")

print("Success! You are logged in")
```

(ix) use while loop to reverse a given number.



```
num = 45222
print(int(str(num)[::-1]))
```

} By string slicing.

Strings in Python

```
name = "AjayRaj"
surname = "Chauhan" # used for multiline string.
gender = 'Male'
print(name + surname + "Gender: " + gender)
```

String indexing

```
name = "Ajay"
print(name[0]) → A
print(name[1]) → j
print(name[2]) → a
print(name[3]) → y
print(name[4]) → IndexError: string index out of range
```

print(name[-1]) → y.

A	j	a	y
0	1	2	3
-4	-3	-2	-1
↓	↓	↓	↓
-4+4	-3+4	-2+4	-1+4

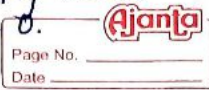
To convert negative index to positive add the length of the string to the negative index (Ajay → length → 4)

String slicing

```
name = "AjayRaj"
print(name[0:2]) # goes from 0 to 1 (gives 1st two characters).
print(name[2:-1]) # goes from [2: 5]
```

```
# print(name[0:7:n]) # skips (n-1) characters.
print(name[0:7:2]) # skips 1 character
print(name[0:6:1]) # skips 0 character # output AaRj (skips j, y, a)
```


print(name[:4]) # Replace the 1st empty number with 0.
same as name[0:4]



print(name[1:]) # Replace second empty no with length
same as name[1:7]. # goes from 1 to 6.

String methods in Python

name = "Harry" # strings are immutable

name[0] = 'R' # You can't do this (TypeError)

a = len(name)
print(a) } Output = 5 (length of the string).

s = "Hello World"
a = len(s)
print(a) # output: 11

print(s.upper(), s) # output: HELLO WORLD Hello World
Changes into upper case but original string is unchanged.

print(s.capitalize()) # Only 1st character is capitalized.

print(s.lower()) # change all characters into lower case

print(s.title()) # 1st character of every word is capitalized.

text = " hello world "

print(text.strip()) # Remove empty spaces from left and right
print(text.rstrip()) # Remove " " from right

print(text.lstrip()) # " " " from left.

text = "Python is fun"

print(text.find("is")) # Output = 7 (Index of first occurrence)

print(text.replace("fun", "awesome"))

Replaces all occurrences of fun with awesome.

text = "Apples, Bananas, Pineapples"
print(text.split(", ")) # convert it into a list

output: ['Apples', 'Bananas', 'Pineapples']

print(", ".join(['Apples', 'Bananas', 'Pineapples']))

Convert the list into a string.

text = "Python 123"
print(text.isalpha()) # output: False (check that all characters are alphabets)
print(text.isdigit()) # " " (check that all characters are digits)

print(text.isalnum()) # output: True (checks that the string is alphabet + numeric string)

print(text.isspace()) # output: False

String formatting

template = "Dear Harry, You are awesome, Take this
1000\$ bag."

```
a = "John"
a1 = 10000
b = "Jack"
b1 = 20000
c = "Marie"
c1 = 300
```

```
st = template.format(a, a1)
print(st)
```

Output: Dear John, You are awesome, Take this 10000\$ bag.

```
print(f"{a} you are awesome and take this {a1}$ bag")
      |
      f-string
```

ord() and chr() → character encoding

print(ord('A')) → output: 65 (gives ascii value)

print(chr(65)) → output: 'A' (gives character value)

(i) Concatenate two strings with space b/w them

```
str1 = "Hello"
str2 = "World"
# print(str1 + " " + str2)
print(str1, str2)
```

(ii) Remove every ~~two~~ second character from the string

```
text = "Python Programming"
print(text[::2])
```

```
* text = "I love Python Programming"
print(text.strip())
print(text.title())
```

```
print(text.count("o")) # Count occurrence of o
```

```
* str = "123abc"
```

```
if str.isalnum() str.isalnum():
    print("Yes string is alphanumeric")
```

```
else:
    print("No string is not alphanumeric")
```

```
* name = "Ajay"
age = 21
```

```
print(f"My name is {name} and I am {age} years old.")
```

```
* name = "Ajay"
age = 21
```

```
print("My name is {} and I am {} years old.".format(name, age))
```

* Count no. of vowels in a given string.

```
sentence = "Coding in Python is fun." → sum = 0
for char in sentence.lower(): vowels = ['a', 'e', 'i', 'o', 'u']
    print(char)
    if char in vowels:
```

```
        sum += 1
print(f"There are {sum} vowels in this sentence")
```


Functions in Python

```
* def average(a, b, c): # function definition.  
    d = (a+b+c)/3.0  
    print(d)
```

average(3, 5, 1) # function call.
average(4, 2, 1)

```
* def average(a, b, c):  
    d = (a+b+c)/3.0  
    return d
```

o1 = average(2, 3, 5)
o2 = average(6, 8, 9)

print(o1)
print(o2)

```
def add(a, b):  
    return a+b
```

c = add(3, 5)
print(c).

a, b → Parameters
3, 5 → arguments.

```
* def add(a, b, plus=0): # Default argument  
    x = a+b+plus  
    return x  
c = add(2, 3)  
print(c)
```

c1 = add(b=5, a=3) → keyword arguments.

Lambda function in Python

```
square = lambda x: x*x  
print(square(4)) # output: 16
```

lambda functions are anonymous inline functions.

```
square = lambda x: x*x  
sum = lambda x, y: x+y.
```

```
''' As good as writing  
def sum(x, y):  
    return x+y '''
```

print(square(3))
print(sum(3, 5))

Recursion in Python

```
''' Fibonacci series = 0 1 1 2 3 5 8...  
fib(0)=0  
fib(1)=1  
fib(n)= fib(n-2) + fib(n-1) '''
```

```
* def fib(n):  
    # Base case of recursion  
    if (n==0 or n==1)  
        return n  
    return fib(n-1) + fib(n-2)  
  
print(fib(6))
```

```
* def factorial n:  
    if (n==1 or n==0):  
        return 1  
    return n*factorial(n-1)
```

print(factorial(5))

Modules and pip in Python

Python provides built-in and third party modules.

```
import math
```

```
print(math.sqrt(16)) # Square root of 16
```

Types of modules

→ Built-in modules

→ External modules → written by us or installed by pip.

list of built-in modules:-

<https://docs.python.org/3/py-modindex.html>

```
* def hello():  
    # Import from another module  
    import mymodule  
    print("Hello World")  
    mymodule.hello()
```

Variable scope and Decoding

In python, variables have scope (where they can be accessed) and lifetime (how long they exist).

Types of scope

(i) local scope (inside a function) → Variables declared inside a function are accessible only within the function.

(ii) Global scope (accessible everywhere) → Variables declared outside any function can be used throughout the program.

```
def sum(a,b):
```

```
    c = a + b # a and b are local variables  
    return c # can't be accessed outside the function
```

```
print(sum(4,6))
```

```
print(c) → Error.
```

```
def sum(a,b):
```

```
    # a and b are local variables
```

```
    c = a + b
```

```
    z = 1 # It creates a local variable z which  
          is destroyed after this function returns.  
    return c.
```

```
z = 8 # z is a global variable (Not inside any function)  
print(z)  
print(sum(4,6))
```

```
* x = 10 # global variable  
def my_func():  
    x = 5 # local variable  
    print(x) # Output = 5
```

```
my_func()  
print(x) # Output = 10 (Global x remains unchanged)
```

Global keyword:- Modify the variable

```
def sum(a,b):
```

```
    c = a + b
```

```
    global z # modify global z
```

```
    z = 0 # This will refer to global z and not create  
           a local variable
```

```
z = 3
```

```
print(sum(3,12))
```

```
print(z) # Output: 0
```

~~Excessive~~ # Excessive use of global is discouraged as it can make debugging harder.

Docstring:- use to document function, classes and modules. In python, they are written in triple quotes.

They are accessible using the `__doc__` attribute

```
def sum(a,b):
```

```
    "This will sum two numbers."
```

```
    c = a + b
```

```
    return c
```

```
print(sum(3,4))
```

```
print(sum.__doc__)
```

```
* def full_name(first, last):
```

```
    return f"{first} {last}"
```

```
print(full_name("Ajay", "Raj"))
```

```
* def calculate_area(length, width=10):
```

```
    return length * width
```

```
print(calculate_area(15, 20))
```

```
print(calculate_area(13))
```

```
* def calculate_area(length, width=10):
```

```
    square = lambda x: x * x
```

```
    list1 = [1, 2, 3, 4, 5]
```

(Print square of all nos. of in the list)

```
print(list(map(square, list1)))
```

```
* def sum_of_digits(n):
```

```
    if n == 0:
```

```
        return 0
```

```
    return n % 10 + sum_of_digits(n // 10)
```

```
# sum of digits of 7532 = 2 + sum of digits of 753
```

```
    = 2 + 3 + sum of digits of 75
```

```
print(sum_of_digits(7532))
```

```
* import math
```

```
b = math.sin(math.radians(90)) # value of sin 90
```

```
print(b)
```

```
* Install and import the requests module and use it to fetch data from "https://api.github.com"
```

```
import requests
```

```
a = requests.get("https://api.github.com")
```

```
print(a.json())
```

```
* Write a function increment() that has local variable counts initialized to 0 and increments it by 1 each time it is called. observe whether the value persists across function calls.
```

```
def increment():
```

```
    counts = 0
```

```
    counts += 1 # local variable
```

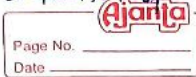
```
    print(counts)
```

```
increment()
```

```
''
```

```
''
```


Lists in Python → ordered, mutable collection of items.



```
marks = [54, 93, 62, 92, 32]
mixed = [43, "hello", false, 42]
```

```
print(marks) → print(marks[0])
print(mixed)   print(marks[1])
print(mixed[1:3])
```

```
print(mixed[4]) # IndexError
```

* marks = [5, 2, 21, 7, 5]

```
marks.append(6) # This will change the original list
print(marks) # [5, 2, 21, 7, 5, 6]
```

```
marks.pop() # Remove last element.
```

```
extra_marks = [53, 23, 32]
marks.extend(extra_marks)
print(marks)
```

```
marks.insert(1, 99) # [5, 99, 21, 7, 5, 6]
marks.remove(21) # [5, 99, 7, 5, 6]
```

```
marks.reverse() # [6, 5, 7, 99, 5]
```

```
marks.sort() # Convert into ascending order.
```

```
marks.count(5)
```

List Comprehension

Create a list containing the table of 5



```
M-I: a = 5
table = []
for i in range(1, 11):
    table.append(5*i)
print(table)
```

```
M-II: # By list comprehension.
table = [5*i for i in range(1, 11)]
print(table)
```

Tuples in Python: ordered, immutable

```
a = (3, 2, 22, 13)
a[3] = 32 # TypeError (because immutable)
```

```
b = (5,) # Creation of tuple with single element
```

tuple unpacking:

```
tu = (3, 2, 45)
```

```
a, b, c = tu
```

```
print(a, b, c) # 3 2 45 (a=3, b=2, c=45)
```

Common tuple methods

* Fixed collections

count, index

- * Faster than lists
- * Used as dictionary keys
- * Safe from unintended modification.

Sets in Python = unordered, unique collection
(no duplicates)

```
s = {1, 2, 2, 11}
```

```
print(s, type(s))
```

```
print(s[2]) # object is not subscriptable  
            (TypeError :- unique collection)
```

```
s.add(32) # use to add elements in sets.  
print(s)
```

```
s.remove(3) # use to remove element
```

```
s.discard(4332) # Remove element only if it is  
                present if not present no error.
```

```
a = {1, 2, 3, 4, 5}
```

```
b = {5, 6, 7}
```

```
c = a.union(b)
```

```
print(c) # {1, 2, 3, 4, 5, 6, 7}
```

```
d = a.intersection(b)
```

```
print(d) # {5}
```

* Used for removing duplicates
* Used in set operations.

```
e = a.difference(b) # (a-b)  
print(e)
```

Dictionaries in Python

Stores key-value pairs and allow fast lookups.

```
marks = {"harry": 34, "Jack": 45, "lily": 35}
```

```
print(marks, type(marks))
```

	for key, value in marks.items():
	print(key, value)

```
print(marks["lily"])
```

```
marks["harry"] = 40 # change value
```

```
print(marks)
```

```
marks["John"] = 49 # adding key-value pair.
```

```
print(marks.keys()) # dict-keys (['harry', 'Jack', 'lily'])
```

```
print(marks.values()) # dict-values ([40, 45, 35])
```

```
marks.pop("lily") # use to remove key-value pair.
```

```
marks.clear() # clears the dictionary.
```

Dictionary Comprehension

```
* table_of_5 = {i: 5*i for i in range(1, 11)}
```

```
print(table_of_5)
```

```
* squares = {x: x**2 in range of (1, 10)}  
print(squares)
```

* Used for fast lookups
* Used to store structured data

* fruits = ["apples", "banana", "cherry"]
 print (fruits [0])
 print (len(fruits))
 fruit[1] = "orange" # replaces banana with orange
 print (fruits)

* list1 = [i for i in range(1,11)]
 print (list1)
 print (list1[0:3]) # Prints first 3 nos
 print (list1[-3:]) # Prints last 3 nos.

* num = [5, 2, 9, 1, 7]
 num.sort()
 print (num)
 num.append(10) # add 10 to num.
 num.remove(10)

* name = ["Alice", "Bob", "Charlie"]
 name.insert(1, "David") # Insert David at index 1
 print (name)

imp * To add / remove element from tuple :-

Coordinates = (10, 20)
 print (coordinates[0])
 print (coordinates[1])
 coordinate[1] = 50 # Not possible (Tuples are immutable)

corList = list (coordinates)
 corList[1] = 50
 coordinates = tuple (corList)

* make a list and remove all duplicate elements using set.

* Given a dictionary of product and their prices, find the product with the highest price.

* write a program to merge two dictionaries into one

* list1 = [1, 2, 4, 78, 4, 9, 11, 2]
 set1 = set (list1)
 list1 = list (set1)
 print (list1)

* products = {"Laptop": 50000, "Mobile": 20000, "Tablet": 30000, "Headphones": 4000 }

max_price = 0
 max_product = ""

for products, price in products.items():
 if price > max_price:
 max_price = price
 max_product = product
 print (max_product, max_price)

* dict1 = {"a": 10, "b": 20}
 dict2 = {"c": 30, "d": 40}

dict1.update (dict2)

print ("Merged Dictionary:", dict1)

Object-oriented programming in Python

Ajanta

Page No.
Date

Advantages:

- Data organisation
- Reusability
- Easier Debugging
- Real world modeling

Principles of OOPS

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

Objects and classes in Python

```
class Employee:  
    company = "HP"
```

```
    def get_salary(self):  
        return 34000
```

```
e = Employee() # object of class Employee  
print(e.get_salary())
```

```
e2 = Employee()  
print(e2.get_salary())  
print(e2.company)
```

Constructors in Python

Ajanta

Page No.
Date

class Employee:

```
    def __init__(self, salary, name, bond):  
        self.salary = salary # create an instance attribute of  
        self.name = name      name salary and assign it  
        self.bond = bond       with salary.
```

```
    def get_salary(self):  
        return self.salary
```

```
e1 = Employee(24000, "John Doe", 4)  
print(e1.get_salary())
```

calculator

try:

```
a = int(input("Enter first no.?:"))
```

```
b = int(input("Enter second no.?:"))
```

```
print("What kind of operation do you want to perform.  
Press + for addition \n Press - for subtraction \n  
Press * for multiplication \n Press / for division")
```

```
o = input("Enter operation")
```

match o:

case "+":

```
    print(f"The result is: {a+b}")
```

case "-":

!

default:

```
    print(f"There was an error")
```

except Exception as e:

```
    print("Enter a valid value of a and b")
```


Decorators in Python

Decorator is a function that takes a function, it creates a new function inside its body (wrapper), then it returns the new function.

```
* def decorator(func):
    def wrapper():
        print("I am about to execute a function...")
        func()
        print("I have executed this function..")
    return wrapper
```

→ @ decorator

```
def say_hello():
    print("Hello")

# say_hello()
f = decorator(say_hello)
f()
```

To replace this

```
""" function f will look like this
def f():
    print("I am about --")
    print("Hello")
    print("I have executed --") """
```

* Decorators with arguments.

```
def repeat(n):
    def decorator(func):
        def wrapper(a): # argument a
            for i in range(n):
                func(a)
        return wrapper
    return decorator
```

```
@ repeat(7)
def say_hello():
    print("Hello! 493")

say_hello("Namy")
```

Uses of Decorators:

logging

Timing

Authentication and Authorization

Caching

Rate Limiting

Input validation

getters and setters in Python.

class Employee:

```
def __init__(self, name, salary):
```

```
    self.name = name
```

```
    self.salary = salary → @ property
```

```
def first_name(self):
```

```
    l = self.name.split(" ")
```

Return first name → return l[0] → @ first_name.setter

```
e = Employee("Jack Doe", 24555)
```

```
print(e.first_name())
```

```
e.set_first_name("John")
```

```
print(e.name)
```

```
def set_first_name(self, first):
```

```
    l = self.name.split(" ")
```

```
    new_name = f'{first} {l[1:]}'
```

```
    self.name = new_name
```

```
print(e.first_name)
```

```
e.first_name = "John"
```

```
print(e.name)
```


Static Methods, class methods and Instance Methods

Page No. _____
Date _____

class Employee:

company = "HP"

def __init__(self, name, salary):

self.name = name

self.salary = salary

e1 = Employee("Jack", 13455)

e1.print_company()

e2 = Employee("Jia", 13455)

e1.change_company()

print(Employee.company)

print(Employee.company)

print(Employee.name) # Error

e1.print_info()

e2.print_info()

print(e2.sum(5, 23))

Instance def print_info(self):

Method. return f"The name is {self.name} and the salary is {self.salary}"

@ static method # static method (method that doesn't require self)
def sum(a, b):
return a + b

@ classmethod # class method
def print_company(cls):
print(cls.company)

@ class method
def change_company(cls, new_company):
cls.company = new_company

Magic / Dunder Methods

eg: __init__, __str__, __add__

Page No. _____
Date _____

Used to customize object creation and initialization
Enable operator overloading

provide string representations of objects (__str__, __repr__)
Control attribute access (__getattr__, __setattr__, __delattr__)
Make objects callable (__call__)

Common magic methods

(i) __init__: used for object initialization.

class Person:

def __init__(self, name, age):

self.name = name

self.age = age

p = Person("Alice", 30)

print(p.name, p.age)

print(str(p))

print(repr(p))

print(len(p))

(ii) __str__: used to return human readable, informal string representation of the object.

(iii) __repr__: use to return unambiguous, official string representation of object.

def __str__(self):

return f"The name is {self.name} and the salary is {self.salary}"

def __repr__(self):

return f"name: {self.name} \n salary: {self.salary}"

def __len__(self):

return len(self.name)

class Vector:

```
def __init__(self, x, y):
```

```
    self.x = x
```

```
    self.y = y
```

```
def __add__(self, other):
```

```
    return Vector(self.x + other.x, self.y + other.y)
```

```
def __sub__(self, other):
```

```
    return Vector(self.x - other.x, self.y - other.y)
```

```
def __mul__(self, scalar):
```

```
    return Vector(self.x * scalar, self.y * scalar)
```

```
def __str__(self):
```

```
    return f"Vector({self.x}, {self.y})"
```

```
v1 = Vector(2, 3)
```

```
v2 = Vector(4, 5)
```

```
v3 = v1 + v2 # calls __add__
```

```
print(v3)
```

```
v4 = v3 - v1 # calls __sub__
```

```
print(v4)
```

```
v5 = v1 * 5
```

```
print(v5)
```

Common operator overloading methods

```
__eq__(==)
```

```
__gt__(>)
```

```
__truediv__(/)
```

```
__ne__(!=)
```

```
__le__(<=)
```

```
__floordiv__(//)
```

```
(1)
```

```
(2)
```

Errors and Exception Handling in Python

* While True: # used for infinite loop

try:

```
a = int(input("Enter no. 1:"))
```

```
b = int(input("Enter no. 2:"))
```

```
print(f"The sum is {a+b}")
```

except:

```
print("Some error occurred!")
```

or

except

Exception as e:

```
print("Some error occurred!", e)
```

except ValueError:

```
print("Please don't perform bad typecasts")
```

except ZeroDivisionError:

```
print("Division by zero not possible")
```

* a = int(input("Enter no. 1:"))

b = int(input("Enter no. 2:"))

if b == 0:

```
raise ValueError("Please don't divide by 0")
```

used to throw error with personalised message.

```
print(f"The division is {a/b}")
```


*

try:

a = 345/0

except Exception as e:

print(e)

else:

print("Hey I am good") # Executed when there is no error in try block

* { a = int(input("Enter a no. "))
b = int(input("Enter another no. "))
def divide(a, b):

try:

c = a/b

print(c) → return c

except Exception as e:

print(e)

return None

finally: # Always executed no matter if try executes or not.
print("This is always executed")

→ a = int(input(" - - - "))

b = int(input(" - - - "))

divide(a, b)

Map, filter and reduce

These are higher order functions.

They operate on iterables (list, tuples)

* numbers = [1, 4, 7, 44, 21]

def square(x):

return x*x

new = list(map(square, numbers))

print(new)

" map function applies a given function to each item of an iterable and returns an iterator that yields the result."

* def is_greater_than_9(x):

if x > 9:

return True

else:

return False

a = [1, 3, 5, 234, 34, 32, 6543, 23, 2, 5, 6, 7, 43]

new = list(filter(is_greater_than_9, a))

print(new)

filter filters the iterable and returns only the no.s greater than 9.

* numbers = [1, 2, 3, 45, 4]

new = list(map(lambda x: x*x, numbers))

print(new)

* from functools import reduce

numbers = [1, 2, 3, 4, 5, 6]

[3, 3, 4, 5, 6] → [6, 4, 5, 6] → [10, 5, 6]
[15, 6] → [21]

```
def sum(a, b):  
    return a + b:
```

```
c = reduce(sum, numbers)  
print(c)
```

Walrus operators in Python (:=)

Use to assign a value to a variable within an expression.

se Case: (i) Conditional Expressions:-

without walrus operator

```
data = input("Enter a value (or 'quit' to exit):")
```

```
while data != "quit":
```

```
    print(f"You entered: {data}")
```

```
    data = input("Enter a value (or 'quit' to exit):")
```

with walrus operator

```
while (data := input("Enter a value (or 'quit' to exit):")):  
    print(f"You entered: {data}")
```

(ii) List comprehensions:-

no.s = [1, 2, 3, 4, 5]

without walrus operator: calculate $x * 2$ twice
results = [x * 2 for x in numbers if x * 2 > 5]

with walrus operator: calculate $x * 2$ only once
results = [y for x in numbers if (y := x * 2) > 5]

args and kwargs in Python.

* def sum(*args)

~~args = args~~ # args will be a tuple of all the values passed to sum.

for item in args:

total += item

return total

```
print(sum(342, 7, 9)) # print(sum(342, 7, 9, 10, 15))
```

* def marks(**kwargs)

for item in kwargs.keys():

print(kwargs[item])

print(f"The marks of {item} is {kwargs[item]}")

```
marks(shubham=34, vikrant=54, jayant=97)
```

kwargs is a dictionary with all the key value pairs which were passed to marks.

* def fun1(*args, **kwargs)

print(args)

print(kwargs)

```
fun1(1, 2, 4, 5, jack=34, jill=32, muneer=32)
```


* class NegativeNumberError (Exception):
pass



try:

```
num = int(input("Enter a number"))
```

```
if num < 0:
```

```
    raise NegativeNumberError("Number cannot be negative")
```

```
result = 45/num
```

```
print(f"The result is {result}")
```

```
except ValueError:
```

```
    print("Error: Please enter a proper number")
```

```
except ZeroDivisionError:
```

```
    print("Error: Cannot divide by 0")
```

```
except NegativeNumberError as e:
```

```
    print("Error: {e}")
```

Read, write and Append files in Python



File I/O in Python: It refers to reading data from and writing data to files.

Steps involving working with files:

- (i) Opening a file
- (ii) Performing operations
- (iii) Closing the files.

* file.txt

Hey this is Ajay and I love Python.

I also work with AI and ML applications.

```
f = open("harry.txt", "r")
```

Open in read mode
rt → read text file

```
content = f.read()
```

rb → read binary file

```
print(content)
```

```
f.close
```

* # John Doe.txt. (Write to a file It should contain data about John Doe)

```
f = open("John Doe.txt", "w")
```

- string = "John Doe is a nice guy. He lives in NYC.

```
f.write(string)
```

His favourite package is python"

f.close() → John Doe.txt file will be created

If the file exists, its content will be overwritten. If it does not exist, a new file will be created.

append mode(a):- opens the file for appending.
Data will be append to the end of the file.
If it does not exist a new file will be created.

Append to an existing file called John Doe.txt

```
f = open("John Doe.txt", "a")
```

```
string = "John Doe works with Pandas and Numpy"
```

```
f.append(string) / f.write(string)  
f.close()
```

Reading from a file:-

try:

```
file = open("myfile.txt", "r")  
content = file.read()  
print(content)  
file.close()
```

except FileNotFoundError:

```
print("File Not found")
```

Reading line by line

try:

```
file = open("file.txt", "r")  
for line in file: # Efficient for large files.  
    print(line.strip()) # Remove newline characters  
file.close()
```

except FileNotFoundError:

```
print("File not found")
```

Another method to read a file

```
with open("file.txt", "r") as f: # Context manager  
    content = f.read()  
    print(content)
```

No need to write f.close() because file is already closed by default when using with syntax

OS and Shutil Modules in Python

```
import os  
a = os.listdir("dir")  
print(a) # will give list of all the files in directory.
```

directory → name (dir)
file 1.txt
file 2.txt

* OS module provides functions for interacting with operating system, such as working with directories and files.

* The shutil module offers higher-level file operations

```
print(os.getcwd()) # Current working directory.
```

```
print(os.path.exists("dir")) # Checks whether a file or directory exists or not
```

```
os.remove("sample.txt") # Use to remove files from directories.
```

```
os.rmdir("dir") # Use to remove empty directories
```

```
path = os.path.join("folder", "subfolder", "file.txt")
```

```
print("Joined Path:", path)
```

Joined path components in a platform-independent way.

import shutil

shutil.rmtree("dir") # It can clear all files from a directory

* shutil.copy("harry.txt", "John.txt") # use to copy a file.

shutil.move("John.txt", "dir/") # Moving file into a directory.

Creating Command line utilities in Python.

import argparse

parser = argparse.ArgumentParser(description="simple Calculator")

parser.add_argument("num1", type=float, help="first no")

parser.add_argument("num2", type=float, help="second no")

parser.add_argument("operation", choices=["add", "sub", "mul", "div"],
help="operation to perform")

args = parser.parse_args()
print(args)

if (args.operation == "add"):

print(f"The result is {args.num1 + args.num2}")

elif
elif
elif

else:

print("some error occurred")

* with open("file.txt", "w") as f:

f.write("learning Python is fun")

with open("file.txt", "r") as f:

print(f.read())

* with open("tasks.txt", "w") as f:

f.write("line1\n")

f.write("line2\n")

f.write("line3\n")

with open("tasks.txt", "a") as f:

f.write("Task completed")

* write a small script count_line.py that takes a filename as input and prints how many lines are in the file.

import sys

def count_lines(filename):

with open(filename) as f:

return len(f.readlines())

if __name__ == "__main__":

filename = sys.argv[1]

num_lines = count_lines(filename)

print(f"There are {num_lines} lines in {filename}")

import sys

def search_word(word, string):

return string.count(word)

if __name__ == "__main__":

filename = sys.argv[1]

words = sys.argv[2]

with open(filename) as f:

string = f.read()

n = search_word(word, string)

print(f'There are {n} occurrences of {word} in the file')

Virtual Environment and Package Management

Virtual Environments: - It is a self contained directory that contains its own python interpreter and libraries.

This means that libraries installed in one virtual environment won't interfere with libraries in another.

To install - pip install virtualenv

Creating a virtual environment:-

python3 -m venv my_env # creates a virtual environment named "my_env"

Windows:- my_env\scripts\Activate.ps1 # To activate the virtual environment.

pip install pandas # install pandas in vE my_env.

pip install requests

pip install numpy==1.20.0 # installs specific version

pip list # list of packages installed in our environment

pip install --upgrade requests # upgrade outdated (name of package) package

pip uninstall requests # uninstall packages (package name)

deactivate # used for deactivating virtual environment

pip freeze # shows all packages with version no.

Page No.
Date

Ajantha

pip freeze > requirements.txt # Create the requirements file

pip install -r requirements.txt # Install packages

Request Modules - Working with APIs

import requests

url = "https://api.github.com/users/octocat"
Example API endpoint

response = requests.get(url)

if response.status_code == 200:
data = response.json() # Parse the JSON response

print(data["name"]) # Access data from JSON

else:
print(f"Error: {response.status_code}")

Make a POST request (for sending data to an API):

data = {"key": "value"}

response = response.post(url, json=data) # sends data as JSON

other HTTP method: put(), delete(), etc.

Regular Expressions in Python

These are powerful tools for pattern matching in string. Python's re module provides supports for regex

Page No.
Date

Ajantha

import re

text = "The quick brown fox jumps over the brown fox"

Search for a pattern

match = re.search("brown", text)

if match:

print("Match found!")

print("Start Index:", match.start())

print("End Index:", match.end())

Find all occurrences of a pattern

matches = re.findall("the", text, re.IGNORECASE)

Case sensitive

print("Matches:", matches)

Replace all occurrences of a pattern

new_text = re.sub("fox", "cat", text)

print("New text:", new_text)

Compile a regex for efficiency (if used multiple times)

pattern = re.compile(r"\b\w+\b") # Matches whole words

words = pattern.findall(text)

print("Words:", words)

{b} - word boundary
{w+} - one more word characters

Multi threading (using threading module)

it is suitable for I/O-bound tasks (eg. waiting for network requests)
install threading module

```
import threading
import time
```

```
def worker(num):
    print(f"Thread {num}: Starting")
    time.sleep(2) # simulate some work
```

```
print(f"Thread {num}: Finishing")
```

```
threads = []
for i in range(3):
    thread = threading.Thread(target=worker, args=(i,))
    thread.append(thread)
    thread.start()
```

```
for thread in threads:
    thread.join() # wait for all threads to finish.
```

```
print("All threads completed!")
```

Who wants to be a millionaire Game

questions = [

["Who is Shahrukh Khan?", "BWF wrestler", "Actor", "Plumber", "Cook", 2]

["Who wrote Romeo and Juliet", "Charles Dickens", "William Shakespeare", "Jane Austen", "Nomer", 2]

prizes = [10000, - , - , - , -] ~~sum = 0~~ i = 0

```
for question in questions:
    print(question[0])
    print(f"a. {question[1]}")
    print(f"b. {question[2]}")
    print(f"c. {question[3]}")
    print(f"d. {question[4]}")
```

check whether the answer is correct or not

```
a = int(input("Enter your answer. \n 1 for a, 2 for b, 3 for c, 4 for d"))
```

```
if (question[5] == a):
    print("Correct answer.")
```

else:

```
print(f"Incorrect, the correct answer was {question[5]}")
print("Better luck next time!")
break sum = prizes[i]
```

```
print(f"You won {prizes[i]}")
i += 1
```


PDF merger → Go to PyPDF and do all other operations. Documentation with Python.

from PyPDF2 import PdfWriter
merger = PdfWriter()

pdfs = []
n = int(input("How many pdfs you want to merge\n"))

for i in range(0, n):
name = input(f"Enter the name of pdf {i+1}:")
pdfs.append(name)

for pdf in pdfs:
merger.append(pdf)

merger.write("merged-pdf.pdf")
merger.close()

News App in Python → News API → Documentation
import requests.

query = "Data Science" # You can enter copy the url.
your own query

api = ""

url = f"

print(url)

[After running a url will be generated → run into browser (Not readable)]
r = requests.get(url)
data = r.json() → in form of JSON formatter (Enable in Chrome)
articles = data["articles"] dict

for article in articles: # for index, article in enumerate(articles)
print(article["title"]) # or print(article["title"], article["url"])
print(article["url"]) # print("\n *** - - - *** \n")

or you can change the:-
query = input("what type of news are you interested in today?")

Click ctrl on the url to open the particular news.

Drink Water Reminder App

import time:
while True: # used for infinite loop.

~~pip~~
pip install plyer # used for desktop notification.

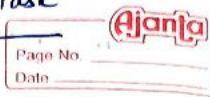
plyer → click on notification

import time
from plyer import notification

while True:
print("Please sip some water!")
notification.notify(title="Please drink some water",
message="You need to drink some water",)

time.sleep(60*60)

Building web applications using Flask



pip install flask

Flask folders → static, template, main.py

Flask minimal application → copy code

from flask import flask

app = flask(__name__)

@app.route("/")

def hello_world():
 return "<p>Hello, World! </p>"

app.run(debug=True)

→ In template

index.html → ! + Enters → gives you bootstrap plate html

↳ to run it on flask.py.

{ change title
head
body }

from flask import flask, render_template
app = flask(__name__)

@app.route("/")
def hello_world():

return render_template("index.html")

app.run(debug=True)

Creating a static site

Flask - static folder → main.py, static, template

main.py

from flask import flask, render_template
app = flask(__name__)

@app.route("/")
def home():
 return render_template("home.html")

@app.route("/services")
def services():
 return render_template("services.html")

@app.route("/contact")
def contact():
 return render_template("contact.html")

@app.route("/about")
def about():
 return render_template("about.html")

app.run(debug=True)

home.html, services.html, contact.html, about.html → create in templates
use bootstrap → copy starter template and bootstrap css and js.

navbars → Click download example → Paste in home.html.

Copy its index.html → Paste it in home.html → in the body.
Check bootstrap documentation

change title and nav. item.

Copy and paste in all html files.
and change the title (add the name in front)
of html file

To highlight paste active in nav-item-
(of the html file you want to highlight) ← nav-link active

Copy button / accordion. etc from bootstrap
and put in in a container in the required
page.

create a container.

```
<div class="container"> → (create <h1> - </h1>
|
| → copy the code for accordion. <p> - - </p>
|
</div>
```

Serving static files in flask
main.py
from flask import flask, render_template
static → 1.zip → bootstrap ex.s.
templates → index.html
↓

```
app = flask(__name__)
@app.route("/")
def hello_world():
    return render_template("index.html")
```

body
<h1> serving static files </h1>
<p> download the template
here:
download </p>

app.run(port=8000, debug=True)

* put the files in static that you want to download